

UNSA

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN DE
AREQUIPA

Facultad de Producción y Servicios
Escuela Profesional de Ingeniería de Sistemas

INFORME FINAL DE PROYECTO

Helicopter Attack 3D Videojuego de Combate Táctico en Unity

Curso: Computación Gráfica, Visión Computacional y Multimedia
Docente: Mag. Rene Alonso Nieto Valencia
Repositorio: KevinCallo/TrabajoFinalCGVCYM
Semestre: 2026-I
Fecha: Julio 2026

Estudiante:

N°	Apellidos y Nombres
1	Callo Ccagiavilca, Kevin Joel

Arequipa — Perú
2026

Índice

1. Introducción y Descripción General del Proyecto	2
2. Enlaces del Proyecto	2
3. Proceso de Desarrollo de la Aplicación	2
4. Tecnologías y Herramientas Empleadas	3
5. Funcionalidades y Arquitectura de Código	3
5.1. Mapeo de Componentes Principales	3
5.2. Esquema de Controles del Usuario	3
6. Desafíos Técnicos y Soluciones Implementadas	3
6.1. Corrección de Autodaño al Disparar	3
6.2. Ajuste del Suelo mediante Raycast	4
6.3. Filtro de Visión Nocturna CCTV	4
7. Lecciones Aprendidas	4
8. Conclusiones	4

1. Introducción y Descripción General del Proyecto

El presente informe documenta la arquitectura, diseño, implementación y resultados del proyecto **Helicopter Attack 3D**, desarrollado para la asignatura de *Computación Gráfica, Visión Computacional y Multimedia* de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de San Agustín de Arequipa (UNSA).

El objetivo principal fue transformar y expandir un prototipo interactivo 3D en un videojuego completo para PC en el motor **Unity 3D**. El usuario pilotea un helicóptero de combate para defender una posición estratégica contra oleadas de tanques enemigos dotados de Inteligencia Artificial (IA).

El sistema integra física de combate aéreo-terrestre, ciclo ambiental acelerado de día y noche, renderizado de cámara de visión nocturna en escala de grises estilo circuito cerrado de televisión (CCTV) mediante un *Shader HLSL* personalizado, y una campaña por oleadas temporizadas de 5 minutos.

2. Enlaces del Proyecto

A continuación se presentan los enlaces oficiales requeridos para la verificación y evaluación del proyecto:

- **Repositorio Oficial de Código Fuente (GitHub):**
<https://github.com/KevinCallo/TrabajoFinalCGVCYM.git>
- **Aplicación Compilada Ejecutable (.exe para Windows):**
<https://github.com/KevinCallo/TrabajoFinalCGVCYM/raw/main/Helicoptero%20Attack.exe>
- **Video Demostrativo de Funcionamiento:**
https://drive.google.com/file/d/1a23ZvTwslw2yfO1-sFzamW_jcmu3jXFA/view?usp=sharing

3. Proceso de Desarrollo de la Aplicación

El desarrollo se llevó a cabo en cuatro fases principales:

1. **Análisis y Refactorización del Control de Entrada:** Se sustituyó el sistema de mandos táctiles móviles (*Joysticks*) por un esquema de entrada unificado para PC (Teclado WASD, Elevación/Descenso con Espacio/Shift/Ctrl/Q/E y apuntado por ratón).
2. **Sistema de Interfaz y Menús:** Implementación del Menú Principal (*MainMenuUI*) con 3 opciones de campaña, slider de volumen persistente en *PlayerPrefs* y Menú de Pausa (*PauseUI*) que congela la simulación (*Time.timeScale = 0*).
3. **Visión Computacional y Shaders de Procesamiento Nocturno:** Desarrollo del componente *NightCameraEffect* y del sombreador *NightVisionShader.shader* en HLSL con compensación de sombras (*Shadow Lifting*) y conmutación por tecla (N o B).

4. **Inteligencia Artificial y Sistema de Oleadas:** Programación del *WaveManager* para organizar los tanques enemigos en 5 oleadas temporizadas con encaje al suelo por Raycast.

4. Tecnologías y Herramientas Empleadas

- **Motor Gráfico:** Unity 3D Engine (C# scripting).
- **Lenguaje de Programación:** C# (.NET Framework).
- **Lenguaje de Sombreado:** Shader HLSL / CG de Unity (Procesamiento de imagen en espacio de pantalla).
- **Motor de Físicas:** Unity Physics 3D (Rigidbody, Physics.Raycast).
- **Control de Versiones:** Git & GitHub.

5. Funcionalidades y Arquitectura de Código

5.1. Mapeo de Componentes Principales

Componente Script	Ubicación	Responsabilidad Principal
PlayerHeli.cs	Scripts/Gameplay/	Control de vuelo del helicóptero, físicas, alternancia de cámara 1ª/3ª persona y disparo.
InputControl.cs	Scripts/Input/	Captura de entradas de teclado y ratón con bloqueo de cursor.
EnemyTank.cs	Scripts/Gameplay/	IA de persecución, orientación de torreta y disparo de cohetes.
WaveManager.cs	Scripts/Gameplay/	Control del temporizador de 5 minutos y distribución de 5 oleadas.
DayNightCycle.cs	Scripts/Gameplay/	Rotación solar continua y niebla ambiental.
NightCameraEffect.cs	Scripts/Gameplay/	Conmutación por tecla N/B y renderizado de interfaz CCTV.
NightVisionShader.shader	Shaders/	Shader HLSL en escala de grises con ganancia de brillo.

Tabla 1: Mapeo de scripts de la arquitectura del proyecto.

5.2. Esquema de Controles del Usuario

6. Desafíos Técnicos y Soluciones Implementadas

6.1. Corrección de Autodaño al Disparar

Problema: Al instanciar las balas y cohetes desde el helicóptero, los proyectiles colisionaban inmediatamente contra el colisionador del propio jugador.

Solución: Se asignó la propiedad `proj.Creator = gameObject;` en `PlayerHeli.cs` y se agregó una verificación de exclusión de daño en `Projectile.Base.cs`.

Acción	Teclas	Descripción
Movimiento Horizontal	W, A, S, D	Mover adelante, atrás y desplazamiento lateral.
Elevación (Subir)	Espacio / E	Empuje vertical ascendente.
Descenso (Bajar)	Shift Izq / Ctrl Izq / Q	Empuje vertical descendente.
Apuntado	Movimiento del Ratón	Control de cámara y dirección de tiro.
Ametralladora	Clic Izquierdo del Ratón	Disparo de ráfaga rápida.
Lanzacohetes	Clic Derecho del Ratón	Disparo de misiles alternados.
Cámara 1ª/3ª Persona	V / C / Tab	Alterna vista de cabina o vista exterior.
Visión Nocturna	N / B	Activa/desactiva modo CCTV en escala de grises.
Menú de Pausa	Escape / P	Congela la partida y abre opciones.

Tabla 2: Esquema unificado de controles del videojuego.

6.2. Ajuste del Suelo mediante Raycast

Problema: Los tanques enemigos en algunas áreas del mapa quedaban suspendidos en el aire.

Solución: En `WaveManager.cs` se implementó la función `Physics.Raycast(pos + Vector3.up * 50f, Vector3.down, out RaycastHit hit)` para ajustar la posición Y exactamente sobre la superficie del terreno (`hit.point.y + 0.1f`).

6.3. Filtro de Visión Nocturna CCTV

Problema: Al caer la noche la escena quedaba excesivamente oscura.

Solución: Se desarrolló el sombreador `NightVisionShader.shader` con una función de compensación de sombras:

```

1 float gray = dot(col.rgb, float3(0.299, 0.587, 0.114));
2 gray = pow(max(0.001, gray), 0.48) * _Brightness;
3 gray = saturate((gray - 0.35) * _Contrast + 0.35 + 0.12);

```

7. Lecciones Aprendidas

1. **Procesamiento de Imagen en Espacio de Pantalla:** La implementación de Shaders para efectos de post-procesamiento (como visión nocturna CCTV) resulta una alternativa altamente eficiente frente al uso de luces en tiempo real.
2. **Gestión Eficiente de Recursos en Unity:** Reutilizar y activar/desactivar tanques pre-colocados mediante el `WaveManager` garantiza una tasa estable de cuadros por segundo (FPS).

8. Conclusiones

El proyecto **Helicopter Attack 3D** fue desarrollado exitosamente cumpliendo con los estándares académicos de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Nacional de San Agustín de Arequipa (UNSA). La aplicación demuestra la integración efectiva de gráficos por computadora, visión computacional aplicada y sistemas multimedia interactivos.